Bashir Agro System
Phase 12B V2: Flutter Farmer App + Field Boundary + GPS Verification Detailed Implementation

PHASE 12B V2 OBJECTIVE

Build the Flutter Farmer Mobile Application connected with the Laravel API from Phase 12A V2.

The application allows farmers to:

- Login securely
- Receive assigned tasks
- View assigned farm boundaries
- Navigate to fields
- Track GPS location
- Complete work
- Upload evidence
- Submit verified completion

SCOPE

This phase is ONLY for Flutter Mobile App.

Do not modify:

- Laravel Admin Panel
- Database structure
- Boundary creation logic

Field boundaries are created by Admin Web and received through API.

SYSTEM ARCHITECTURE

Admin Panel

↓

Laravel API

↓

PostgreSQL + PostGIS

↓

Flutter Farmer App

Flutter receives:

- Farmer information
- Tasks
- Farm boundary polygon
- Instructions
- Submission requirements

TECHNOLOGY STACK

Flutter + Dart

Maps:
Mapbox Maps Flutter

GPS:
Geolocator

API:
Dio/http

Storage:
Hive or SQLite

Notifications:
Firebase Cloud Messaging

State management:
Riverpod / Provider / Bloc

PROJECT STRUCTURE

lib/

core/
api/
storage/
utils/

features/

auth/
dashboard/
tasks/
farm_map/
submission/

profile/
notifications/

models/

services/

widgets/

AUTHENTICATION

Flow:

Farmer enters login details

↓

Flutter sends API request

↓

Laravel validates

↓

Sanctum token returned

↓

Token stored securely

↓

Dashboard opens

Implement:

- Login
- Logout
- Token expiry handling

DASHBOARD

Display:

- Today's tasks
- Pending tasks
- Completed tasks

- Notifications
- Farmer profile

TASK MODULE

Task List:

Show:

- Task name
- Farm name
- Deadline
- Status

Task Details:

Show:

- Instructions
- Assigned field
- Field map
- Required photos

FIELD BOUNDARY MAP

Important:

The farmer does NOT create boundaries.

Admin creates boundaries.

Flutter receives polygon data through API.

Display:

- Field polygon
- Current farmer location
- Map marker

Technology:

Mapbox Maps Flutter

GPS TRACKING

Capture:

- Latitude
- Longitude
- Accuracy
- Timestamp

Example:

{
latitude:23.8103,
longitude:90.4125,
accuracy:8
}

Send location data to backend.

TASK COMPLETION FLOW

Open Task

↓

View Field Boundary

↓

Navigate to Field

↓

Start Work

↓

Complete Activity

↓

Capture Photos

↓

Capture GPS

↓

Submit

SUBMISSION DATA

Send:

- assignment_id
- notes
- photos
- latitude
- longitude
- accuracy
- timestamp

GPS VERIFICATION

Flutter does NOT decide validity.

Process:

Flutter GPS

↓

Laravel API

↓

PostGIS

↓

Check point inside polygon

YES:
Accept submission

NO:
Reject or flag

OFFLINE SUPPORT

Agricultural areas may have poor connectivity.

Implement:

- Local database
- Pending queue
- Automatic sync
- Retry mechanism

NOTIFICATIONS

Use Firebase Cloud Messaging.

Events:

- New task assigned
- Deadline reminder
- Submission approved
- Submission rejected

SECURITY

Mobile:

- Secure token storage
- API authentication
- Permission handling

Camera:

- Permission request
- File validation

GPS:

- Accuracy checking
- Timestamp validation

TESTING

Authentication:

✓ Login

✓ Logout

✓ Token handling

Maps:

✓ Polygon display

✓ Farmer location

✓ Map loading

Tasks:

✓ Receive tasks

✓ View details

Submission:

✓ Camera

✓ Upload

✓ GPS

Offline:

✓ Save

✓ Sync

END TO END TEST

Admin creates farmer

↓

Admin creates field boundary

↓

Admin assigns task

↓

Farmer logs in

↓

Farmer receives task

↓

Farmer opens map

↓

Farmer reaches field

↓

GPS verification

↓

Submit evidence

↓

Admin reviews

ANTIGRAVITY MASTER INSTRUCTION

Implement PHASE 12B V2 ONLY.

Goal:

Build Bashir Agro Farmer Flutter Application with field boundary support.

Implement:

1. Flutter setup.

2. Authentication.

3. Dashboard.

4. Task list and details.

5. Mapbox polygon display.

6. GPS tracking.

7. Task completion.

8. Photo evidence upload.

9. API integration.

10. Offline synchronization.

11. Firebase notifications.

Rules:

- Do not modify Admin Panel.
- Do not create boundaries in Flutter.
- Use API provided boundaries.
- Backend/PostGIS performs final verification.

After completion report:

- Flutter files created
- Packages added
- API endpoints used
- Screens completed
- Testing results

Run:

flutter test

Stop after Phase 12B V2 completion.